TP INTEGRADOR

Análisis de sentimientos de opiniones sobre películas

Seminario de Actualización · Tecnicatura Superior en Ciencia de Datos e IA

IFTS N.º 18 · Ciclo lectivo 2026

Integrantes

Lucas Suárez · Eric Silvestri · Walter Willich

Entrega: 16 de octubre de 2026

Aplicación

https://tp-integrador-sentimientos.streamlit.app/

Repositorio

https://github.com/Lukit4as/portfolio-seminario

Informe técnico: selección y comparación de modelos preentrenados, pruebas de inferencia, errores, mediciones y despliegue.

A. El problema

El objetivo es analizar una opinión breve sobre una película y estimar el sentimiento expresado. La entrada es texto escrito por el usuario; la salida es una etiqueta y un puntaje de confianza del modelo. Es una demostración educativa, no una herramienta definitiva para decisiones importantes.

Las reglas manuales basadas en palabras como “excelente” o “terrible” pueden fallar con sinónimos, negaciones, ironía, jerga y frases mixtas. Por ejemplo, “Yeah, great movie... I almost fell asleep” incluye una expresión positiva literal, aunque el sentido global es negativo. Los modelos preentrenados aprenden patrones lingüísticos, pero también cometen errores.

B. El modelo

Se compararon dos modelos usando las mismas ocho frases:
distilbert/distilbert-base-uncased-finetuned-sst-2-english y **cardiffnlp/twitter-xlm-roberta-base-sentiment-multilingual**. Se eligió DistilBERT para la aplicación porque cargó de forma estable con Transformers y ofrece una salida binaria clara. XLM-R se utilizó como comparación y puede devolver las etiquetas positive, neutral y negative.

DistilBERT es un clasificador de sentimiento en inglés ajustado con SST-2. Tiene aproximadamente 67 millones de parámetros y pesos de alrededor de 268 MB. Devuelve POSITIVE o NEGATIVE, por lo que no puede expresar una clase neutral y no garantiza resultados fiables en español. XLM-R utiliza una variante multilingüe de XLM-RoBERTa; el carácter multilingüe no garantiza que resuelva correctamente la ironía, la jerga o todos los matices culturales. Para detalles exactos de corpus, idiomas y limitaciones se deben consultar las model cards de ambos checkpoints.

Resultados sobre las ocho frases

El criterio esperado se definió para este experimento y no representa una verdad universal. Los puntajes son los devueltos por cada modelo; no equivalen a una probabilidad calibrada de que la interpretación sea correcta.

Frase	Esperado	DistilBERT etiqueta / score	XLM-R etiqueta / score
This movie was absolutely amazing!	Positivo	POSITIVE · 0,9999 ✓	positive · 0,9855 ✓
I hated every minute of this movie.	Negativo	NEGATIVE · 0,9996 ✓	negative · 0,9136 ✓
La película fue excelente y me encantó.	Positivo	POSITIVE · 0,9832 ✓	positive · 0,9854 ✓
La película fue aburrida y demasiado larga.	Negativo	NEGATIVE · 0,9936 ✓	negative · 0,9611 ✓
Yeah, great movie... I almost fell asleep.	Negativo	NEGATIVE · 0,9850 ✓	positive · 0,9856 ✗
This movie was sick!	Positivo	NEGATIVE · 0,9998 ✗	negative · 0,8912 ✗
Meh.	Neutral	NEGATIVE · 0,5080 ✗	neutral · 0,6126 ✓
The acting was good, but the story was terrible.	Negativo	NEGATIVE · 0,9929 ✓	negative · 0,8975 ✓

Resultado: ambos modelos coincidieron con el criterio esperado en 6 de 8 entradas (75 %). La muestra es pequeña y no permite estimar el rendimiento general.

En la frase irónica, XLM-R predijo positivo, posiblemente atendiendo a “great movie”. En “This movie was sick!”, ambos predijeron negativo, aunque “sick” puede usarse como elogio informal. En “Meh.”, DistilBERT forzó una clase negativa porque no contempla neutralidad; XLM-R devolvió neutral. La aplicación no entrena ni ajusta pesos: realiza inferencia con modelos ya entrenados.

C. Dónde corre cada cosa

El código está en GitHub, dentro de la carpeta TP_Integrador. Streamlit Community Cloud obtiene el repositorio y ejecuta app.py. Transformers y PyTorch corren en el servidor; al iniciar, el modelo descarga sus pesos desde Hugging Face Hub y los mantiene en el entorno o en caché. La inferencia de la aplicación publicada se realiza en el servidor de Streamlit.

Usuario / navegador Escribe la opinión y ve el resultado	**Streamlit Community Cloud** Ejecuta app.py y calcula la predicción	**Hugging Face Hub** Aloja los pesos del modelo
Envía el texto →	← Devuelve etiqueta y score	→ Descarga de pesos al iniciar

GitHub guarda código, configuración, README y notebook, no los pesos del modelo. El navegador envía el texto al servicio y recibe una respuesta. Se eligió Streamlit Community Cloud porque permite publicar una interfaz Python conectada al repositorio. Render y Hugging Face Spaces son alternativas; para compararlas deben considerarse memoria, tamaño de los pesos, arranque en frío, cuotas y límites gratuitos. No se afirma que una plataforma sea superior en todos los escenarios.

D. Pipeline e InferenceClient: mediciones y decisión

Se registraron dos tipos de medición para distinguir el costo de inicialización de la inferencia con los modelos ya cargados. En una prueba anterior, la primera predicción con pipeline tardó aproximadamente 7,68 segundos e incluía carga e inicialización; una predicción posterior tardó alrededor de 0,049 segundos. Esos valores dependen del entorno y de la caché y no son directamente comparables con una tanda de varias frases.

En la sesión actual, con los modelos ya cargados, se procesaron ocho frases por modelo:

Modelo	Tiempo total (8 frases)	Promedio por frase	Condición
DistilBERT	0,3697 s	0,0462 s	Modelo ya cargado
XLM-RoBERTa	0,6581 s	0,0823 s	Modelo ya cargado

La salida de pipeline fue una lista que contiene un diccionario. Se accede a la etiqueta con resultado[0]["label"] y al puntaje con resultado[0]["score"]. En las pruebas, type(resultado) devolvió list y type(resultado[0]) devolvió dict. Para la llamada individual, el pipeline es una instancia de TextClassificationPipeline; la llamada recibe el texto como argumento y devuelve la lista de resultados.

También se intentó la misma tarea con huggingface_hub.InferenceClient. El proveedor respondió HTTP 402 Payment Required porque no quedaban créditos de inferencia. No se compraron créditos ni se inventó una respuesta o un tiempo de API, por lo que no se completó una comparación de latencia remota. Pipeline ejecuta la inferencia en el entorno de la aplicación y requiere memoria para cargar el modelo; InferenceClient envía la entrada a un proveedor remoto y depende de conectividad, credenciales y cuota. Para esta app se eligió pipeline porque funcionó sin depender de créditos de API, siempre que la plataforma tenga recursos suficientes.

E. Lo que se rompió: herramientas y errores reales

Se probaron modelos del Hub con Transformers y se intentó el acceso remoto mediante huggingface_hub.InferenceClient. Durante el trabajo, un checkpoint inicial de CardiffNLP presentó errores de carga/tokenizador; se probó una variante multilingüe que sí cargó. También hubo un HTTP 401 al validar un token; se revisó la credencial y se evitó guardar tokens en el código.

El proveedor de InferenceClient respondió HTTP 402 Payment Required: no quedaban créditos para usar Inference Providers. Se documentó el mensaje y se continuó con pipeline sin comprar créditos. GitHub también rechazó un push por detectar tokens en el historial (GH013). Se limpió el notebook, se reemplazó el commit y se verificó que el patrón del token ya no apareciera antes de publicar. Estos errores muestran que la autenticación, los permisos, los créditos y la seguridad del historial son problemas diferentes.

Un token es una credencial personal. No debe incluirse en el informe, README, notebook ni repositorio. Si se expone, hay que revocarlo y reemplazarlo; borrarlo en un commit posterior no elimina el secreto de commits anteriores.

F. Despliegue, límites y conclusiones

La aplicación está publicada en Streamlit Community Cloud. El enlace permite escribir una opinión y recibir etiqueta y puntaje. El primer acceso puede tardar por arranque en frío o descarga del modelo; los servicios gratuitos pueden suspender temporalmente la instancia y tienen límites de recursos.

No se realizaron pruebas con 1.000 usuarios simultáneos, por lo que no se afirma que la app soporte esa carga. Una instancia que carga el modelo en memoria puede quedarse sin recursos o responder lentamente. Para escalar habría que medir concurrencia, memoria y latencia, añadir monitoreo y evaluar instancias con más recursos o inferencia remota.

El proyecto demuestra selección de modelos preentrenados, comparación con entradas comunes, inspección de resultados, medición de inferencia, construcción de una interfaz y despliegue. Ambos modelos coincidieron con el criterio esperado en 6 de 8 frases, pero tuvieron errores diferentes. Como próximos pasos se ampliaría la muestra, se probarían más condiciones y se evaluaría un modelo multilingüe en la aplicación.

Vocabulario técnico y uso de IA

Instancia: objeto pipeline creado al cargar el modelo. **Función:** cargar_modelo() y la llamada a pipeline. **Clase:** TextClassificationPipeline. **Método:** la llamada al objeto pipeline para ejecutar la clasificación. **Atributos/claves de salida:** "label" y "score". **Parámetro:** nombre definido en una función, como model o truncation cuando corresponda. **Argumento:** valor concreto pasado al parámetro, por ejemplo el texto analizado. **Retorno:** lista que contiene un diccionario con etiqueta y puntaje.

Se utilizó ChatGPT como apoyo para diagnosticar errores, estructurar el código de la interfaz y organizar el informe. El código se ejecutó en Codespaces y la aplicación se probó manualmente; los errores y resultados incluidos se basan en las salidas registradas. Los integrantes deben poder explicar y verificar las decisiones.

Aplicación: https://tp-integrador-sentimientos.streamlit.app/
Repositorio: https://github.com/Lukit4as/portfolio-seminario
Notebook: TP_Integrador/TP_Integrador.ipynb